

fTOP Curriculum

Overview

This curriculum provides a comprehensive, phased approach to mastering the software ecosystem used in High Energy Physics (HEP) experiments. It progresses from foundational tools through to advanced, experiment-specific frameworks, culminating in hands-on work with real experimental data. Each phase builds upon the previous, ensuring a coherent and practical learning experience.

Phase 1: Foundations and Core Tools

Description

This phase establishes the essential technical foundation for all HEP data analysis work. Students are introduced to ROOT — the industry-standard data analysis framework developed at CERN — alongside the statistical principles that underpin all experimental measurements in particle physics. Python-based tooling for modern HEP analysis is also introduced, bridging traditional C++ workflows with contemporary data science practices.

Topics

ROOT Framework ROOT is an object-oriented framework written in C++ and widely used across HEP experiments for data storage, processing, and visualization. Students learn to navigate ROOT files, create and manipulate histograms, perform fits, and write analysis macros in both C++ and Python.

Statistical Methods Foundation A rigorous grounding in statistical inference as applied to particle physics. Topics include probability theory, parameter estimation (maximum likelihood, least squares), hypothesis testing, confidence intervals, and systematic uncertainties. These concepts are essential for interpreting any experimental result.

Python for HEP Analysis Modern HEP analysis increasingly relies on Python-based tools. This section introduces `uproot` for reading ROOT files natively in Python and `awkward-array` for efficiently handling the complex, nested, variable-length data structures characteristic of particle physics event data.

Phase Outcomes

By the end of Phase 1, students will be able to:

- Open, read, and manipulate ROOT files using both C++ macros and Python scripts
- Produce, style, and interpret histograms and fit results within ROOT
- Apply core statistical concepts — likelihood functions, hypothesis tests, confidence intervals — to physics problems
- Read HEP data into Python using `uproot` and perform array-based

manipulations with `awkward-array` - Understand the structure of typical HEP event data and how it is stored

Resources

Resource	Description
ROOT Primer (Beginner's Guide)	Complete introduction to ROOT with hands-on examples
ROOT Manual	Comprehensive official documentation covering all aspects
First Steps with ROOT	Getting started guide
ROOT Tutorials	Collection of C++ macros and Python scripts for learning
ROOT Reference Documentation	Complete API reference for advanced usage
Statistical Data Analysis — Glen Cowan	Classic textbook with exercises and solutions
Glen Cowan's Statistical Methods Lectures	Comprehensive lecture series with problem sheets
Statistical Methods for Data Analysis in Particle Physics — Luca Lista (2023)	Most recent edition, includes machine learning chapter
Data Analysis in Particle Physics — Cesare Bini (PDF)	Lecture notes from Sapienza Università
Uproot Documentation	Getting started with modern Python ROOT I/O
Uproot Tutorial — Mason Proffitt	Complete hands-on tutorial
Awkward Array Documentation	Manipulating complex nested data structures
PyHEP 2021 Uproot and Awkward Tutorial	Advanced topics including $H \rightarrow ZZ \rightarrow 4\mu$ analysis

Phase 2: Advanced Statistical Analysis

Description

Building on the statistical foundations of Phase 1, this phase introduces professional-grade statistical modeling tools used in published HEP results. Students learn to construct complex probability models using RooFit, perform frequentist and Bayesian inference with RooStats, and apply multivariate analysis (MVA) techniques using ROOT's built-in machine learning toolkit, TMVA.

Topics

RooFit and RooStats RooFit is a toolkit for modeling probability density functions (PDFs) and performing unbinned maximum likelihood fits — the standard method for extracting physics results from data. RooStats extends this with tools for hypothesis testing, limit-setting, and discovery significance calculations, forming the statistical backbone of most major HEP analyses.

Machine Learning with TMVA TMVA (Toolkit for Multivariate Analysis) provides a unified interface for training and applying machine learning classifiers within the ROOT environment. Students learn to implement Boosted Decision Trees (BDTs), neural networks, and other discriminants to separate signal from background, a core technique in modern particle physics searches.

Phase Outcomes

By the end of Phase 2, students will be able to: - Build and fit probability density function models using RooFit - Construct statistical tests, compute p-values and confidence limits using RooStats - Train and evaluate multivariate classifiers (BDTs, MLPs) with TMVA - Understand the workflow of a signal extraction analysis from data model construction through to result interpretation - Apply these tools to realistic analysis scenarios typical of CMS or ATLAS analyses

Resources

Resource	Description
RooFit Manual (ROOT)	Official documentation for probability density functions
RooStats Workbook	Documentation with practical examples
RooStats Workbook GitHub	Includes 2018 Terascale Statistics School materials
RooFit Basics Tutorial (CMS Combine)	Practical introduction to fitting
CMS Statistics Exercise	Hands-on RooStats tutorial
NA62 RooFit/RooStats Tutorials	Practical statistical treatment examples
TMVA Manual (ROOT)	Complete guide to machine learning in ROOT
TMVA Users Guide (PDF)	Comprehensive documentation
TMVA Classification Tutorial	Example code and documentation
MultiVariate Analysis Tutorial — Francesco Gravili (PDF)	Step-by-step guide to MVA

Phase 3: Simulation Tools and Frameworks

Description

Simulation is the cornerstone of HEP experiment design and data interpretation. This phase covers the three principal simulation tools in modern HEP: Geant4 for full, physics-accurate detector simulation; Delphes for fast parametric simulation suitable for phenomenological studies; and DD4hep for structured, experiment-agnostic detector geometry description.

Topics

Geant4 — Full Detector Simulation Geant4 is the industry-standard Monte Carlo toolkit for simulating the passage of particles through matter. It handles all relevant physics processes — electromagnetic, hadronic, and optical — and is used by virtually every major HEP experiment worldwide. Students learn to construct detector geometries, define materials and physics lists, and process simulated particle interactions.

Delphes — Fast Parametric Simulation While Geant4 is highly accurate, it is computationally expensive. Delphes provides a fast, parameterized simulation of detector response, ideal for phenomenological studies, feasibility studies, and detector optimization. It reads generator-level events and applies smearing functions to mimic detector effects, outputting analysis-ready data.

DD4hep — Detector Description Toolkit DD4hep provides a unified, modular approach to describing detector geometry that is compatible with both Geant4 and Delphes workflows. It separates the detector description from the simulation engine, enabling reuse across different software frameworks and experiments. This is essential for future collider projects such as FCC and ILC.

Phase Outcomes

By the end of Phase 3, students will be able to:

- Build and run a basic Geant4 simulation application with a custom detector geometry and physics list
- Interpret Geant4 output and understand how simulated hits relate to detector response
- Run fast simulation workflows using Delphes with custom detector cards
- Describe a detector geometry using DD4hep and interface it with simulation engines
- Understand the trade-offs between full and fast simulation and when each is appropriate

Resources

Resource	Description
Getting Started with Geant4	Introduction and installation guide
Geant4 Documentation	Complete user guides and reference manuals

Resource	Description
Geant4 Application Developers Guide	How to build simulation applications
Geant4 Physics Reference Manual	Detailed physics interactions documentation
Geant4 Tutorials	Official tutorial materials
Geant4 Advanced Examples	Realistic experimental applications
Delphes Website	Official documentation and resources
Delphes GitHub Repository	Source code and quick start guide
DELPHES 3 Paper (arXiv:1307.6346)	Technical description of the framework
Delphes Tutorial — Jan Eysermans (PDF)	FCC workshop tutorial
Software Tutorial: LCIO and Delphes	Snowmass Energy Frontier tutorial
DD4hep GitHub Repository	Main source code and documentation
DD4hep User Manual (PDF)	Complete technical documentation
DD4hep Introduction	General overview and architecture
DD4hep Code Reference	API documentation
DD4hep Paper (arXiv)	Technical publication

Phase 4: Event Data Models and Processing Frameworks

Description

Modern HEP experiments require robust, standardized data models and processing frameworks to manage the enormous volume and complexity of collision event data. This phase introduces EDM4hep — the common event data model for future collider experiments — and the Gaudi framework, the algorithmic processing backbone used by LHCb, ATLAS, and future experiment software stacks.

Topics

EDM4hep — Common Event Data Model EDM4hep defines a standardized, experiment-agnostic representation of physics objects (tracks, clusters, jets, particles) produced during event reconstruction. Built on top of the Podio I/O framework, it enables interoperability between different simulation, reconstruction, and analysis tools. Understanding EDM4hep is essential for anyone working in the Key4hep or FCC software ecosystem.

Gaudi Framework Gaudi is a C++ software framework designed for building HEP data processing applications. It provides a clean, modular architecture separating algorithms, services, and data objects, allowing complex processing pipelines to be composed from reusable components. Originally developed for LHCb, it now underpins many experiment software stacks including the FCC software chain.

Phase Outcomes

By the end of Phase 4, students will be able to: - Navigate and work with EDM4hep data structures for reconstructed physics objects - Understand the role of Podio as the I/O layer underpinning EDM4hep - Write and configure Gaudi algorithms for custom event processing tasks - Understand the architecture of a Gaudi-based application (algorithms, tools, services, and the event store) - Connect simulation output to downstream analysis through standardized data models

Resources

Resource	Description
EDM4hep GitHub Repository	Source code and basic documentation
EDM4hep API Reference	Doxygen documentation
EDM4hep Tutorial in Key4hep	Practical introduction
EDM4hep YAML Definition	Complete data model specification
EDM4hep Paper (ICHEP 2022)	Technical presentation
EDM4hep and Podio (CDS)	Detailed technical documentation
Gaudi Documentation Homepage	Main documentation portal
Gaudi Introduction	User guide and tutorial overview
Gaudi Architecture	Framework design and concepts
Gaudi Services	Framework services documentation
Gaudi Developers Guide	Complete reference

Phase 5: Key4hep and FCC Software Stacks

Description

This phase brings together the tools and concepts from previous phases into complete, integrated software stacks targeting future collider experiments. Key4hep is the common software framework for next-generation experiments (FCC, ILC, CLIC, CEPC), integrating Gaudi, EDM4hep, DD4hep, Geant4, and Delphes into a coherent, deployable ecosystem. The FCC software stack is examined as the primary concrete application of Key4hep.

Topics

Key4hep Framework Key4hep is the overarching software initiative that integrates all the component tools studied in previous phases into a unified, turnkey framework. It provides the glue layer that allows DD4hep geometry to drive Geant4 simulation, whose output is stored in EDM4hep format, processed through Gaudi algorithms, and analyzed with Python-based tools. Students learn to set up, configure, and extend a full Key4hep workflow.

FCC Software Stack The Future Circular Collider (FCC) software stack is built entirely on Key4hep and provides a complete environment for FCC-ee, FCC-hh, and FCC-he studies. It includes configuration for fast simulation with Delphes (via k4SimDelphes), full simulation with Geant4 (via k4SimGeant4), and a dedicated analysis framework (FCCAnalyses) built on ROOT's RDataFrame. This phase gives students practical experience with a realistic future-experiment software environment.

Phase Outcomes

By the end of Phase 5, students will be able to: - Set up and configure a full Key4hep software environment - Run end-to-end simulation and reconstruction workflows within Key4hep - Use k4SimGeant4 for full simulation and k4SimDelphes for fast simulation in the FCC context - Write analysis code using the FCCAnalyses framework and RDataFrame - Understand how the different components (DD4hep, Gaudi, EDM4hep, Geant4, Delphes) integrate within a unified software stack

Resources

Resource	Description
Key4hep Documentation	Central documentation hub
Getting Started with Key4hep	Installation and setup
Key4hep Tutorials GitHub	Complete tutorial collection
Key4hep Introduction Tutorial	Prerequisites and basic introduction
Developing Key4hep Software	Developer documentation
k4FWCore GitHub	Core components for Gaudi-based framework
k4SimGeant4 GitHub	Geant4 simulation components
FCC Software Homepage	Main entry point and links
FCC Tutorials	Complete tutorial collection
FCC Software Stack Overview	Software ecosystem description
FCCSW GitHub Repository	High level repository
FCC Software Introduction Tutorial	Getting started with FCCSW
FCC Prerequisites	Platform requirements and setup
FCC Software Forum	Support and discussions
FCC Delphes Tutorial	Fast simulation with Delphes
FCCAnalyses Problems and Solutions	Common issues and fixes

Phase 6: Experiment-Specific Frameworks

Description

While phases 1–5 focus on general-purpose and future-experiment frameworks, this phase provides exposure to the software used in currently operating large-scale experiments. The ATLAS Athena framework is studied as a representative example of a mature, production-grade experiment software system, illustrating how the concepts from previous phases manifest in real experimental contexts.

Topics

ATLAS Athena Framework Athena is the software framework underpinning all ATLAS data processing, from online event selection through offline reconstruction and analysis. It is built on Gaudi and uses ATLAS’s own event data model (xAOD). Students learn to navigate the Athena software environment, understand the xAOD data format, and perform analysis-level tasks using the ATLAS Analysis Base release. This provides crucial context for understanding what production HEP software at scale looks like.

Phase Outcomes

By the end of Phase 6, students will be able to: - Set up an ATLAS software environment and navigate the Athena framework - Understand the xAOD event data model and access physics objects from reconstructed data - Write basic analysis algorithms within the Athena/AnalysisBase framework - Appreciate how a large-scale experiment manages software releases, versioning, and data formats - Relate Athena’s architecture back to the Gaudi concepts studied in Phase 4

Resources

Resource	Description
Athena Introduction	Framework basics
ATLAS Software Documentation	Complete documentation portal
ATLAS Analysis Software Tutorial	Hands-on xAOD analysis
Athena I/O and xAOD EDM	Data format and event data model
Athena Releases Documentation	Version management

Phase 7: Practical Application with Real Data

Description

This phase moves from tools and frameworks to genuine scientific practice, applying everything learned in previous phases to real collision data made

publicly available by CERN. Students perform end-to-end analyses on CMS and ATLAS open datasets, including the reproduction of landmark results such as the Higgs boson discovery. This phase consolidates all prior learning in a scientifically meaningful context.

Topics

CERN Open Data Analysis CERN makes real collision data from CMS and ATLAS publicly available through the CERN Open Data Portal. Students perform complete analysis workflows: accessing data, applying selection criteria, modeling signal and background, and extracting a physics result. The flagship exercise is an independent rediscovery of the Higgs boson in the $H \rightarrow ZZ$ channel using 2011–2012 CMS data, mirroring the actual discovery analysis.

Phase Outcomes

By the end of Phase 7, students will be able to: - Access and navigate datasets from the CERN Open Data Portal - Understand the data formats used by CMS (AOD/MiniAOD) and ATLAS - Perform a complete, publication-style analysis on real collision data - Reproduce the Higgs boson discovery signal from public CMS data - Critically evaluate analysis choices including selection criteria, systematic uncertainties, and statistical interpretation

Resources

Resource	Description
CERN Open Data Portal	Main repository for open datasets
CMS Open Data Guide for Education	Educational resources
CMS HEP Tutorial	One-week course with real CMS data
ATLAS Open Data	ATLAS open data resources
Higgs Discovery Example	Re-discover the Higgs boson
About CMS Open Data	Understanding CMS data formats
About ATLAS Open Data	Understanding ATLAS data formats

Phase 8: Community Training and Best Practices

Description

This phase situates the student within the broader HEP software community and addresses software engineering practices specific to physics research. It covers the HEP Software Foundation’s training ecosystem and the foundational software engineering skills — version control, scripting, C++ proficiency — that are expected of any working HEP physicist or software developer.

Topics

HEP Software Foundation Training The HEP Software Foundation (HSF) coordinates community-wide training initiatives through partnerships with IRIS-HEP and other organizations. Students engage with the structured HSF training curriculum covering Unix shell, Git, Python, and software carpentry, establishing the collaborative software development habits essential in large experimental collaborations.

Software Engineering for Physicists C++ remains the primary language of HEP software infrastructure. This section provides a systematic treatment of C++ as applied in a HEP context: memory management, templates, object-oriented design patterns, and best practices for writing maintainable, efficient physics software. The emphasis is on practices that are specifically relevant to the challenges encountered in large HEP codebases.

Phase Outcomes

By the end of Phase 8, students will be able to:

- Use Git effectively for version control in a collaborative development context
- Write well-structured, maintainable C++ code following HEP community conventions
- Navigate and contribute to large, community-maintained software repositories
- Understand the HSF training landscape and how to engage with community resources
- Apply software carpentry skills (Unix shell, scripting, testing) in a physics workflow

Resources

Resource	Description
HSF Training Center	Central hub for HEP software training
HSF Training GitHub	All training modules and materials
HSF/IRIS-HEP Software Basics Training	Unix, Git, Python fundamentals
IRIS-HEP Training Program	Complete training vision
C++ in Particle Physics Guide	Best practices overview
Introduction to C++ for Particle Physicists	Course with HEP examples
C++ Programming for Physicists (PDF)	Comprehensive guide
Education and Training for Software Developers in Particle Physics	Pedagogical perspectives

Phase 9: Advanced and Specialized Topics

Description

The final phase consolidates and extends expertise in the two areas most central to cutting-edge HEP analysis: advanced ROOT functionality and sophisticated statistical methods. Students explore features of ROOT that go beyond basic usage — particularly RDataFrame for modern, parallel, declarative analysis — and deepen their statistical grounding with advanced inference techniques used in flagship analyses.

Topics

Advanced ROOT Features RDataFrame is ROOT’s modern, functional-style analysis interface that enables lazy evaluation and automatic parallelization of analysis workflows. TMVA SOFIE (Speedy Operators for Fast Inference Engines) allows trained neural networks from external frameworks (TensorFlow, PyTorch, ONNX) to be deployed directly within ROOT for fast inference. These represent the current state of the art in ROOT-based analysis.

Advanced Statistical Concepts This section revisits statistical methods at a deeper level, covering topics such as the CLs method for limit setting, the profile likelihood ratio test statistic, systematic uncertainty treatment via nuisance parameters, and Bayesian inference with posterior sampling. These are the techniques used in the most precise measurements and sensitive searches published by ATLAS and CMS.

Phase Outcomes

By the end of Phase 9, students will be able to: - Write efficient, parallelized analyses using RDataFrame - Deploy trained neural networks within ROOT using TMVA SOFIE - Apply the CLs method and profile likelihood ratio for setting exclusion limits - Incorporate nuisance parameters to handle systematic uncertainties in a likelihood model - Critically read and understand the statistical sections of ATLAS and CMS publications

Resources

Resource	Description
ROOT Data Analysis Framework	RDataFrame, advanced histograms, fitting
TMVA SOFIE	Fast inference for neural networks
Statistical Methods Lecture Notes — Glen Cowan (PDF)	Advanced statistical concepts

Summary: Curriculum Roadmap

Phase	Focus Area	Key Technologies
1	Foundations and Core Tools	ROOT, Python, uproot, awkward-array
2	Advanced Statistical Analysis	RooFit, RooStats, TMVA
3	Simulation Tools	Geant4, Delphes, DD4hep
4	Event Data Models and Processing	EDM4hep, Podio, Gaudi
5	Key4hep and FCC Software	Key4hep, k4SimGeant4, FCCAnalyses
6	Experiment-Specific Frameworks	ATLAS Athena, xAOD
7	Practical Application with Real Data	CERN Open Data, CMS/ATLAS datasets
8	Community Training and Best Practices	HSF, C++, Git, software engineering
9	Advanced and Specialized Topics	RDataFrame, TMVA, SOFIE, advanced statistics